BreaQ 2026 — Quantum Hacking Track

Challenge 1: *Trust No Input* — Prerequisites

Team: **GravityFalls**

24 October 2026

1 Quantum basics (just enough)

Concept	What you need to know
Qubit	A quantum bit. It starts at 0, and when measured it gives 0 or 1.
Superposition	Before measurement a qubit can be "both 0 and 1" with some probabilities. The Hadamard gate `h` creates this.
Gates	Operations on qubits. Only three are needed: `x` (NOT), `h` (Hadamard, which undoes itself when applied twice) and `cx` (CNOT, which flips the target if the control is 1).
Measurement	Reads a qubit as a plain 0 or 1. The result is random unless the circuit makes it certain.
Shots	A circuit is run many times (here 512). The **counts** say how often each outcome appeared.
Deterministic vs. noisy	On an ideal simulator a well-built circuit can give the **same answer every shot**. Results spread over many keys would suggest noise; all 512 shots on one key means no noise.

2 The Bernstein–Vazirani circuit

Only the idea is needed, not the math:

- A secret 4-bit string (the fingerprint) is hidden in the circuit as CNOTs: one CNOT from qubit i to a helper qubit wherever bit i is 1.
- Hadamard → CNOTs → Hadamard → measure gives back the secret string **with certainty**.
- So the fingerprint can be **read from the diagram**: CNOTs on q_0 and q_2 mean `1010`. If the circuit is right, the counts are always exactly the fingerprint.

3 Qiskit specifics

Bit order. This is the most important concept in the challenge.

- Qubits are numbered q_0, q_1, q_2, q_3, top to bottom in the diagram.
- Count keys are printed with q_0 **on the right**: $q_3\, q_2\, q_1\, q_0$.
- To get the fingerprint in slot order, **reverse the key**: `0101` → `1010`.

Counts dictionary. `{'0101': 512}` maps each outcome to its number of shots. The **dominant key** is the most frequent one: `max(counts, key=counts.get)`.

Statevector. Computes a circuit's exact probabilities without running shots. Remove measurements first with `qc.remove_final_measurements(inplace=False)`. Probability 1.0 means the circuit gives that answer with certainty, which proves the circuit is correct.

The service's tools.

Function	Purpose
`service.execute(case_id)`	Runs a real case and returns counts, report and decision.
`service.get_manifest(case_id)`	Shows the metadata, including `compatibility_map`.
`service.get_circuit(case_id)`	Returns the circuit, to draw it or check it with Statevector.
`service.simulate_adapter(fp, map)`	Test bench: any fingerprint under any map. Returns canonical and reported values.

4 Permutations and counting (the math part)

Permutation. A reordering. $[1, 0, 2, 3]$ lists the four positions in a new order. There are $4! = 24$ permutations of four items.

How the map is applied.

$$\text{reported}[j] = \text{canonical}\big[\,\text{map}[j]\,\big]$$

Position j of the output takes the bit from position map$[j]$ of the input. Example: canonical `1010` with map $[1, 0, 2, 3]$ gives $c_1c_0c_2c_3 =$ `0110`.

Hamming weight. The number of 1s in a string; `1010` has weight 2.

- A permutation only moves bits around, so it **never changes the weight**.
- Therefore a target is reachable from a source **only if both have the same number of** 1s. This is the core of Phase E: `1010` $\rightarrow$ `0110` is possible, `1010` $\rightarrow$ `1110` is impossible.

Counting (for the extension).

- With w ones out of 4, the permutations that leave the string unchanged are those that only shuffle 1s among 1s and 0s among 0s: $w!\,(4-w)!$ of them.
- So the number that **change** it is $24 - w!\,(4-w)!$. For `1010`: $24 - 2! \cdot 2! = 20$.
- The number of distinct strings reachable is $\binom{4}{w}$; for $w = 2$ that is 6.

5 Security concepts

Concept	Meaning here
Threat model	Written answers to: what is being protected, against whom, and what can they touch?
Trust boundary	Where data from outside enters. The manifest crosses it (imported, could be edited); the registry and circuit do not.
Trusted vs. untrusted input	Trusted: registry, circuit, raw counts. Untrusted: manifest metadata. Untrusted data must never decide security outcomes.
Authoritative representation	The one true form of a value. Here it is the **canonical** $q_0 \dots q_3$ **fingerprint**; display formats are secondary.
Presentation vs. security semantics	A legacy map may change how something is *shown*, but must not change a *decision*. The bug is that it does.
Validation ≠ safety	The service checks that the map "is a valid permutation", but every malicious map is a valid permutation. Checking the *format* does not check whether the input should have *authority*.
False reject vs. false accept	False reject: a valid asset is refused (the customer's complaint). False accept: a wrong asset is approved. The same bug allows **false accepts**, the more serious attack.
Root-cause layer	Which layer is at fault: hardware (noise), circuit construction, or application. Here it is the **application**.
Defense in depth	Extra protections such as signing the manifest help, but the real fix is removing the map from the decision.

6 Experimental method

The rules say pointing at the bad line is not enough; the bug must be **proven with experiments**.
Falsifiable claim. A statement that could be proven wrong, e.g. "Changing only X while holding Y fixed changes Z."
Controlled experiment. Change one variable (the map), keep everything else fixed (fingerprint, circuit, seed), and observe. If the output changes, that variable is the cause.
Localization. Check each stage of the pipeline (circuit → counts → report). The first wrong stage is the fault.
Minimal reproducer. The smallest example that shows the bug: one fingerprint, two maps, same quantum result, different reports.
Positive and negative tests. Positive: the right fingerprint gives `MATCH`. Negative: a wrong one gives `MISMATCH`. Test fingerprints *not* in the demo file, to prove the fix is not hard-coded.
Metamorphic testing (extension). Check a property that must always hold without knowing the right answer, e.g. "changing the map must never change the decision."

7 Python you will use

```
key[::-1]                          # reverse a string
max(counts, key=counts.get)        # most frequent key in a dict
from itertools import permutations, product
permutations(range(4))             # all 24 orderings
product("01", repeat=4)            # all 16 four-bit strings
sorted(a) == sorted(b)             # same chars -> same Hamming weight
s.count("1")                       # Hamming weight
format(int(s, 2) ^ 0b1111, "04b")  # flip all bits (handy for negative tests)
```

8 Which concepts each phase needs

Phase	Concepts
Warm-up	Bit order
A: Triage	Counts, dominant key, determinism, threat model, trusted vs. untrusted
B: Localize	Reading the BV circuit, Statevector, pipeline stages, controlled experiment
C: Reproducer	`simulate_adapter`, minimal reproducer
D: Fix	Authoritative representation, presentation vs. security, positive and negative tests
E: Exploit	Permutations, how the map is applied, Hamming weight, false accept
Extension	Counting formula, validation $\neq$ safety, signing and defense in depth, metamorphic tests

The four essentials. Bit order, how the map shuffles bits, Hamming weight, and trusted vs. untrusted input. Everything else supports those.